VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 6: MongoDB Basic commands

Branch/ Class: B. Tech(CSE).
Faculty Name: Prof. S. Gopikrishnan
Student name: B. Aditya Suhas

Date: 26-02-2026
School: SCOPE
Reg. no.:23BCE8879

1. Use MongoDB to implement the following DB operations

   1. Create a database called 'vehicles' and *write* a MongoDB query to select database as "vehicles".

```
mongosh> use vehicles
|
| db.createCollection("two_wheelers", { capped: true, size: 5242880, max: 5000 })
| db.createCollection("four_wheelers")
```

   2. Write a MongoDB query to display all the databases.

```
veichle> show dbs
admin      40.00 KiB
animal     80.00 KiB
config     92.00 KiB
local      40.00 KiB
vehicles   80.00 KiB
```

   3. Create a collection called 'two_wheelers'. (use capping) and Create a collection called 'four_wheelers'.

```
| db.four_wheelers.insertMany([
|   { vehicle_name: "Swift", model: "own", category: "car", variants: ["vxi", "petrol"], manufacturer: "Maruti", performance: 8, timestamp: "2023", price: 7
00000 },
|   { vehicle_name: "Thar", model: "own", category: "car", variants: ["diesel", "petrol"], manufacturer: "Mahindra", performance: 9, timestamp: "2022", pric
e: 1500000 },
|   { vehicle_name: "Eicher Pro", model: "commercial", category: "heavy truck", variants: ["diesel"], manufacturer: "Eicher", performance: 7, timestamp: "20
21", price: 2500000 },
|   { vehicle_name: "Tata Ace", model: "commercial", category: "mini truck", variants: ["diesel", "cng"], manufacturer: "Tata", performance: 8, timestamp: "
2024", price: 500000 },
|   { vehicle_name: "City", model: "own", category: "car", variants: ["vxi", "petrol"], manufacturer: "Honda", performance: 9, timestamp: "2023", price: 120
0000 }
| ])
```

```
switched to db vehicles
vehicles> db.createCollection("two_wheelers", { capped: true, size: 5242880, max: 5000 })
| db.createCollection("four_wheelers")
```

   4. Add 5 two-wheeler details to the collection named 'two_wheelers'. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
mongosh> use vehicles
|
| db.createCollection("two_wheelers", { capped: true, size: 5242880, max: 5000 })
| db.createCollection("four_wheelers")
|
| db.two_wheelers.insertMany([
|   { bike_name: "Activa 6G", model: "gearless", category: "110cc", colors_available: ["red", "black"], manufacturer: "Honda", performance: 8, timestamp: "2
023", price: 75000 },
|   { bike_name: "Pulsar 150", model: "gear", category: "150cc", colors_available: ["black", "blue"], manufacturer: "Bajaj", performance: 7, timestamp: "202
2", price: 110000 },
|   { bike_name: "Jupiter", model: "gearless", category: "125cc", colors_available: ["white", "blue"], manufacturer: "TVS", performance: 9, timestamp: "2024
", price: 85000 },
|   { bike_name: "Duke 200", model: "gear", category: "200cc", colors_available: ["orange", "white"], manufacturer: "KTM", performance: 9, timestamp: "2023"
  price: 190000 },
|   { bike_name: "Splendor Plus", model: "gear", category: "100cc", colors_available: ["silver", "black"], manufacturer: "Hero", performance: 6, timestamp:
"2021", price: 72000 }
| ])
```

5. Add 5 four-wheeler details to the collection named 'four_wheelers'. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
db.four_wheelers.insertMany([
  { vehicle_name: "Swift", model: "own", category: "car", variants: ["vxi", "petrol"], manufacturer: "Maruti", performance: 8, timestamp: "2023", price: 7
0000 },
  { vehicle_name: "Thar", model: "own", category: "car", variants: ["diesel", "petrol"], manufacturer: "Mahindra", performance: 9, timestamp: "2022", pric
: 1500000 },
  { vehicle_name: "Eicher Pro", model: "commercial", category: "heavy truck", variants: ["diesel"], manufacturer: "Eicher", performance: 7, timestamp: "20
1", price: 2500000 },
  { vehicle_name: "Tata Ace", model: "commercial", category: "mini truck", variants: ["diesel", "cng"], manufacturer: "Tata", performance: 8, timestamp: "
024", price: 500000 },
  { vehicle_name: "City", model: "own", category: "car", variants: ["vxi", "petrol"], manufacturer: "Honda", performance: 9, timestamp: "2023", price: 120
000 }
])
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
db.two_wheelers.find()
db.four_wheelers.find()

db.two_wheelers.find({}, { bike_name: 1, price: 1, _id: 0 })
db.four_wheelers.find({}, { vehicle_name: 1, price: 1, _id: 0 })
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database.

```
db.two_wheelers.find({}, { bike_name: 1, price: 1, _id: 0 })
db.four_wheelers.find({}, { vehicle_name: 1, price: 1, _id: 0 })

db.two_wheelers.find({ manufacturer: "Honda" })

db.four_wheelers.find({ variants: "diesel" })

db.two_wheelers.find({ performance: { $gt: 5 } }, { bike_name: 1, category: 1, manufacturer: 1, _id: 0 })
db.four_wheelers.find({ performance: { $gt: 5 } }, { vehicle_name: 1, category: 1, manufacturer: 1, _id: 0 })
[
  { vehicle_name: 'Swift', category: 'car', manufacturer: 'Maruti' },
  { vehicle_name: 'Thar', category: 'car', manufacturer: 'Mahindra' },
  {
    vehicle_name: 'Eicher Pro',
    category: 'heavy truck',
    manufacturer: 'Eicher'
  },
  {
    vehicle_name: 'Tata Ace',
    category: 'mini truck',
    manufacturer: 'Tata'
  },
  { vehicle_name: 'City', category: 'car', manufacturer: 'Honda' }
]
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
vehicles> db.two_wheelers.find({ manufacturer: "Honda" })
[
  {
    _id: ObjectId('69a078c16b36e671527c2907'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '110cc',
    colors_available: [ 'red', 'black' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: '2023',
    price: 75000
  }
]
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
vehicles> db.four_wheelers.find({ variants: "diesel" })
[
  {
    _id: ObjectId('69a078c16b36e671527c290d'),
    vehicle_name: 'Thar',
    model: 'own',
    category: 'car',
    variants: [ 'diesel', 'petrol' ],
    manufacturer: 'Mahindra',
    performance: 9,
    timestamp: '2022',
    price: 1500000
  },
  {
    _id: ObjectId('69a078c16b36e671527c290e'),
    vehicle_name: 'Eicher Pro',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Eicher',
    performance: 7,
    timestamp: '2021',
    price: 2500000
  },
  {
    _id: ObjectId('69a078c16b36e671527c290f'),
    vehicle_name: 'Tata Ace',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel', 'cng' ],
    manufacturer: 'Tata',
    performance: 8,
    timestamp: '2024',
    price: 500000
  }
]
vehicles>
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
vehicles> db.two_wheelers.find({ performance: { $gt: 5 } }, { bike_name: 1, category: 1, manufacturer: 1, _id: 0 })
| db.four_wheelers.find({ performance: { $gt: 5 } }, { vehicle_name: 1, category: 1, manufacturer: 1, _id: 0 })
[
  { vehicle_name: 'Swift', category: 'car', manufacturer: 'Maruti' },
  { vehicle_name: 'Thar', category: 'car', manufacturer: 'Mahindra' },
  {
    vehicle_name: 'Eicher Pro',
    category: 'heavy truck',
    manufacturer: 'Eicher'
  },
  {
    vehicle_name: 'Tata Ace',
    category: 'mini truck',
    manufacturer: 'Tata'
  },
  { vehicle_name: 'City', category: 'car', manufacturer: 'Honda' }
]
vehicles>
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called 'animal' and *write* a MongoDB query to select database as 'animal'.

```
vehicles> use animal
```

2. Write a MongoDB query to display all the databases.

```
veichle> show dbs
admin      40.00 KiB
animal     80.00 KiB
config     92.00 KiB
local      40.00 KiB
vehicles   80.00 KiB
```

3. Create a collection called 'wild_animals'.(use capping) and Create a collection called 'domestic_animals'.

```
db.createCollection("wild_animals", { capped: true, size: 5242880, max: 1000 })
db.createCollection("domestic_animals")
```

4. Add 5 wild_animal details to the collection named 'wild_animals'. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.wild_animals.insertMany([
|   { animal_name: "Lion", nature: "harm", favorite_foods: ["meat", "deer"], care_taker_name: "ADITYA", "life span": 15, timestamp: new Date(), expenses: 50
00 },
|   { animal_name: "Tiger", nature: "harm", favorite_foods: ["meat"], care_taker_name: "SUHAS", "life span": 12, timestamp: new Date(), expenses: 6000 },
|   { animal_name: "Elephant", nature: "harmless", favorite_foods: ["leaves"], care_taker_name: "BANDI", "life span": 60, timestamp: new Date(), expenses: 8
000 },
|   { animal_name: "Cheetah", nature: "harm", favorite_foods: ["rabbits"], care_taker_name: "ADITYA", "life span": 10, timestamp: new Date(), expenses: 4500
 },
|   { animal_name: "Monkey", nature: "harmless", favorite_foods: ["fruits"], care_taker_name: "SUHAS", "life span": 20, timestamp: new Date(), expenses: 200
0 }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a081086b36e671527c291b'),
    '1': ObjectId('69a081086b36e671527c291c'),
    '2': ObjectId('69a081086b36e671527c291d'),
    '3': ObjectId('69a081086b36e671527c291e'),
    '4': ObjectId('69a081086b36e671527c291f')
  }
```

5. Add 5 domestic-animal details to the collection named 'domestic_animals'. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
| db.domestic_animals.insertMany([
|   { animal_name: "Dog", gender: "male", favorite_foods: ["meat"], animal_petname: "Rocky", "life span": 12, timestamp: new Date(), expenses: 1500 },
|   { animal_name: "Cat", gender: "female", favorite_foods: ["fish"], animal_petname: "Kitty", "life span": 15, timestamtimestamp: new Date(), expenses: 100
0 },
|   { animal_name: "Cow", gender: "female", favorite_foods: ["grass"], animal_petname: "Gauri", "life span": 20, timestatimestamp: new Date(), expenses: 300
0 },
|   { animal_name: "Rabbit", gender: "male", favorite_foods: ["carrots"], animal_petname: "Bunny", "life span": 8, timestimestamp: new Date(), expenses: 500
 },
|   { animal_name: "Parrot", gender: "female", favorite_foods: ["seeds"], animal_petname: "Mithu", "life span": 50, timetimestamp: new Date(), expenses: 300
 }
| ])
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
db.wild_animals.find()
db.domestic_animals.find()
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
db.wild_animals.find({}, { animal_name: 1, expenses: 1, _id: 0 })
db.domestic_animals.find({}, { animal_name: 1, expenses: 1, _id: 0 })
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
db.domestic_animals.find({ "life span": 20 })
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
animal> db.wild_animals.find({ care_taker_name: "ADITYA" })
[
  {
    _id: ObjectId('69a081086b36e671527c291b'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'ADITYA',
    'life span': 15,
    timestamp: ISODate('2026-02-26T17:21:12.884Z'),
    expenses: 5000
  },
  {
    _id: ObjectId('69a081086b36e671527c291e'),
    animal_name: 'Cheetah',
    nature: 'harm',
    favorite_foods: [ 'rabbits' ],
    care_taker_name: 'ADITYA',
    'life span': 10,
    timestamp: ISODate('2026-02-26T17:21:12.884Z'),
    expenses: 4500
  }
]
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
animal> db.wild_animals.find({ "life span": { $gt: 5 } }, { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 })
| db.domestic_animals.find({ "life span": { $gt: 5 } }, { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 })
[
  { animal_name: 'Dog', favorite_foods: [ 'meat' ], expenses: 1500 },
  { animal_name: 'Cat', favorite_foods: [ 'fish' ], expenses: 1000 },
  { animal_name: 'Cow', favorite_foods: [ 'grass' ], expenses: 3000 },
  { animal_name: 'Rabbit', favorite_foods: [ 'carrots' ], expenses: 500 },
  { animal_name: 'Parrot', favorite_foods: [ 'seeds' ], expenses: 300 },
  { animal_name: 'Dog', favorite_foods: [ 'meat' ], expenses: 1500 },
  { animal_name: 'Cat', favorite_foods: [ 'fish' ], expenses: 1000 },
  { animal_name: 'Cow', favorite_foods: [ 'grass' ], expenses: 3000 },
  { animal_name: 'Rabbit', favorite_foods: [ 'carrots' ], expenses: 500 },
  { animal_name: 'Parrot', favorite_foods: [ 'seeds' ], expenses: 300 }
]
animal>
```